

1 Oriented prime-multiset ratios

Source: PrimeTensor/Ratio.lean

What this file establishes

A positive rational magnitude is represented as an *ordered pair* of prime multisets — a numerator and a denominator carried separately, never combined. Multiplication is componentwise, inversion swaps the two components, and division is multiplication by a swap.

Two consequences shape everything downstream. First, exponents stay in $\mathbb{N}$: the usual encoding of a positive rational as a $\mathbb{Z}$ -valued exponent vector over the primes is replaced by a pair of $\mathbb{N}$ -valued ones, and what would be negation of an exponent is exchange of orientation. Second, comparison is by cross-multiplication, so neither subtraction nor division appears in the definitions of equality and order.

1.1 The representation

Definition 1.1 (Prime ratio, `PrimeTensor.PrimeRatio`). `PrimeRatio` is the structure with fields

`upper : PrimeMultiset,      lower : PrimeMultiset.`

```
structure PrimeRatio where
  upper : PrimeMultiset
  lower : PrimeMultiset
```

Notation 1.2. For $a : \text{PrimeRatio}$ write

$$|a| \equiv \frac{\text{eval}(a.\text{upper})}{\text{eval}(a.\text{lower})} \in \mathbb{Q}_{>0}$$

for the positive rational a denotes. This is well defined without a side condition: `PrimeTensor/Prime.lean` proves $1 \leq \text{eval}(b)$ for every bag, so the denominator is at least 1 and in particular never zero. Nothing in the file mentions $|a|$ — it is the reading against which the definitions below are stated.

Definition 1.3 (Operations). The declarations `PrimeRatio.one`, `mul`, `inv` and `ratio`:

$$\begin{aligned} \text{one} &\equiv \langle 1, 1 \rangle, \\ a \cdot b &\equiv \langle a.\text{upper} \cdot b.\text{upper}, a.\text{lower} \cdot b.\text{lower} \rangle, \\ a^{-1} &\equiv \langle a.\text{lower}, a.\text{upper} \rangle, \\ \text{ratio}(a, b) &\equiv a \cdot b^{-1}. \end{aligned}$$

The instances `One`, `Mul` and `Inv` give the first three their usual notation.

```
def one : PrimeRatio := ⟨1, 1⟩

def mul (a b : PrimeRatio) : PrimeRatio :=
  ⟨a.upper * b.upper, a.lower * b.lower⟩

def inv (a : PrimeRatio) : PrimeRatio := ⟨a.lower, a.upper⟩
```

```

def ratio (a b : PrimeRatio) : PrimeRatio := mul a (inv b)

instance : One PrimeRatio := (one)
instance : Mul PrimeRatio := (mul)
instance : Inv PrimeRatio := (inv)

```

Under Notation 1.2 these are the expected operations: $|a \cdot b| = |a| |b|$, $|a^{-1}| = |a|^{-1}$ and $|\text{ratio}(a, b)| = |a|/|b|$.

Design note 1.4. Inversion is a swap, and this is the file’s stated reason for the two-field representation. Identifying a positive rational with its exponent vector over the primes, the usual encoding needs exponents in $\mathbb{Z}$, because inverting negates them. Splitting the vector into a positive part and a negative part — upper and lower — keeps both in $\mathbb{N}$, and inversion becomes exchange rather than negation. No signed arithmetic enters, which matters because PrimeMultiset has no additive inverse to offer.

Remark 1.5 (The representation is not normalised). Distinct terms denote the same rational: $\langle 1, 1 \rangle$ and $\langle 2, 2 \rangle$ both denote 1, and no cancellation is performed by mul. So propositional equality of PrimeRatio is finer than equality of denoted magnitude, and the relation of Definition 1.6 is what one must use instead. This is where the file differs from PrimeTensor/Prime.lean: there the representation was quotiented, so that reordering became equality. Here no quotient is taken — the file declares no Setoid and no Quotient — and Same is carried explicitly by downstream statements.

1.2 Equality and order without subtraction

Definition 1.6 (Same, PrimeTensor.PrimeRatio.Same).

$$\text{Same}(a, b) \equiv (\text{eval}(a.\text{upper}) \cdot \text{eval}(b.\text{lower}) = \text{eval}(b.\text{upper}) \cdot \text{eval}(a.\text{lower})).$$

Definition 1.7 (Lt, PrimeTensor.PrimeRatio.Lt).

$$\text{Lt}(a, b) \equiv (\text{eval}(a.\text{upper}) \cdot \text{eval}(b.\text{lower}) < \text{eval}(b.\text{upper}) \cdot \text{eval}(a.\text{lower})),$$

and instance : LT PrimeRatio makes $a < b$ mean Lt(a, b).

```

def Same (a b : PrimeRatio) : Prop :=
  a.upper.eval * b.lower.eval = b.upper.eval * a.lower.eval

/-- Strict order by positive cross-product, with no subtraction. -/
def Lt (a b : PrimeRatio) : Prop :=
  a.upper.eval * b.lower.eval < b.upper.eval * a.lower.eval

```

Proposition 1.8 (Both relations say what they should). *For all $a, b : \text{PrimeRatio}$,*

$$\text{Same}(a, b) \iff |a| = |b|, \quad \text{Lt}(a, b) \iff |a| < |b|.$$

Proof. Write $u_a = \text{eval}(a.\text{upper})$ and $l_a = \text{eval}(a.\text{lower})$, and likewise for b . By Notation 1.2, l_a and l_b are at least 1, hence strictly positive. Multiplying the equation $u_a/l_a = u_b/l_b$ through by the positive quantity $l_a l_b$ gives $u_a l_b = u_b l_a$, and dividing back recovers it; the same computation with $<$ in place of $=$ is valid because multiplying an inequality of rationals by a positive quantity preserves its direction. $\square$

The point of the two definitions is what they avoid rather than what they say. Comparing u_a/l_a with u_b/l_b directly would require division; comparing $u_a l_b - u_b l_a$ with 0 would require subtraction on $\mathbb{N}$, which truncates. Cross-multiplication needs neither, and it stays inside $\mathbb{N}$ where the carrier lives. Positivity of the denominators, inherited from `one_le_eval`, is exactly the hypothesis that makes the reformulation faithful.

Remark 1.9 (What is not proved here). The file states `Same` and `Lt` but proves nothing about them: there is no reflexivity, symmetry or transitivity lemma for `Same`, no transitivity or irreflexivity for `Lt`, and no lemma relating either to `mul` or `inv`. Nor are the monoid laws for `mul` stated, though they follow componentwise from the corresponding laws in `PrimeTensor/Prime.lean`. Compare `PrimeBag.Same`, which does come with its three equivalence lemmas and a `Setoid`. Only the four evaluation lemmas of Section 1.4 are proved.

1.3 Multiplicative neighbourhoods

Definition 1.10 (Within, `PrimeTensor.PrimeRatio.Within`). For $\delta, a, b : \text{PrimeRatio}$,

$$\text{Within}(\delta, a, b) \equiv (\text{ratio}(a, b) < \delta) \wedge (\text{ratio}(b, a) < \delta).$$

```
def Within (δ a b : PrimeRatio) : Prop :=
  ratio a b < δ ∧ ratio b a < δ
```

This is the multiplicative replacement for $|a - b| < \varepsilon$: both $|a|/|b|$ and $|b|/|a|$ are required to fall below $|\delta|$, which pins the two magnitudes to a common annulus. Taking both directions is what makes the condition two-sided without a subtraction or an absolute value, and it makes the relation symmetric in a and b by construction, since the conjunction is symmetric under exchanging them.

Proposition 1.11 (The tolerance must exceed one). *Within(δ, a, a) holds if and only if $\text{eval}(\delta.\text{lower}) < \text{eval}(\delta.\text{upper})$, that is, if and only if $|\delta| > 1$. In particular the relation is reflexive for such δ and holds for no pair at all when $|\delta| \leq 1$.*

Proof. Put $x = \text{ratio}(a, a) = a \cdot a^{-1}$. Unfolding Definition 1.3,

$$x.\text{upper} = a.\text{upper} \cdot a.\text{lower}, \quad x.\text{lower} = a.\text{lower} \cdot a.\text{upper},$$

so by `PrimeMultiset.mul_comm` these two multisets are equal, and in particular $\text{eval}(x.\text{upper}) = \text{eval}(x.\text{lower}) =: E$ with $E \geq 1$. Definition 1.7 then reads $E \cdot \text{eval}(\delta.\text{lower}) < \text{eval}(\delta.\text{upper}) \cdot E$, which since $E > 0$ is equivalent to $\text{eval}(\delta.\text{lower}) < \text{eval}(\delta.\text{upper})$. Both conjuncts of `Within( $\delta, a, a$ )` are this same condition.

For the last claim, note that $|\delta| \leq 1$ forces $\text{ratio}(a, b) < \delta$ and $\text{ratio}(b, a) < \delta$ to fail together: their denoted magnitudes are mutually inverse positive rationals, so at least one of them is $\geq 1 \geq |\delta|$. $\square$

So $|\delta| > 1$ is the multiplicative reading of $\varepsilon > 0$, and $\delta = 1$ is the degenerate tolerance, not the exact one. The file states neither fact; both are recorded here because the analogy with an additive ε otherwise suggests the opposite convention.

1.4 Evaluation lemmas

Lemma 1.12 (`PrimeTensor.PrimeRatio.upper_mul`). $\text{eval}((a \cdot b).\text{upper}) = \text{eval}(a.\text{upper}) \cdot \text{eval}(b.\text{upper})$, and *symmetrically* `lower_mul` for the lower components.

Proof. By Definition 1.3 the projection $(a \cdot b).\text{upper}$ is definitionally $a.\text{upper} \cdot b.\text{upper}$, so after that change of view the goal is `PrimeMultiset.eval_mul` applied to the two upper components. The lower case is the same argument on the other field. Both are tagged `@[simp]`. $\square$

Lemma 1.13 (`inv_upper`, `inv_lower`). $(a^{-1}).\text{upper} = a.\text{lower}$ and $(a^{-1}).\text{lower} = a.\text{upper}$.

```
@[simp] theorem inv_upper (a : PrimeRatio) : (a⁻¹).upper = a.lower := rfl
@[simp] theorem inv_lower (a : PrimeRatio) : (a⁻¹).lower = a.upper := rfl
```

Proof. Both are `rfl`. Inversion is a structure literal whose fields are the projections of a in the other order, and projecting a literal reduces definitionally, so each side is the same term. Note these are stated about the multisets themselves, whereas Lemma 1.12 is stated about their evaluations: nothing is lost, because `mul` acts on the fields while `inv` merely moves them. $\square$

Summary of the correspondence

Lean declaration	Kind	Here
<code>PrimeTensor.PrimeRatio</code>	structure	Def. 1.1
<code>PrimeRatio.one</code>	definition	Def. 1.3
<code>PrimeRatio.mul</code>	definition	Def. 1.3
<code>PrimeRatio.inv</code>	definition	Def. 1.3
<code>PrimeRatio.ratio</code>	definition	Def. 1.3
<code>One, Mul, Inv</code>	instances	Def. 1.3
<code>PrimeRatio.Same</code>	definition	Def. 1.6
<code>PrimeRatio.Lt</code>	definition	Def. 1.7
<code>LT PrimeRatio</code>	instance	Def. 1.7
<code>PrimeRatio.Within</code>	definition	Def. 1.10
<code>PrimeRatio.upper_mul</code>	simp thm	Lem. 1.12
<code>PrimeRatio.lower_mul</code>	simp thm	Lem. 1.12
<code>PrimeRatio.inv_upper</code>	simp thm	Lem. 1.13
<code>PrimeRatio.inv_lower</code>	simp thm	Lem. 1.13

Every declaration of `PrimeTensor/Ratio.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition; it is almost entirely definitions, with four evaluation lemmas. Propositions 1.8 and 1.11, and Notation 1.2, are stated for the reader and are not declarations of the file.